



**Due Date: 23:59 pm on Wednesday, May 1st, 2024**

## Image Classification with Custom CNN and Pre-trained EfficientNet

In this assignment, you will explore image classification using both a custom Convolutional Neural Network (CNN) architecture and a pre-trained EfficientNet model. The goals of this assignment are as follows;

1. Understand CNN architectures and build a model from scratch and train on the dataset provided.
2. Understand and implement transfer learning using a pre-trained EfficientNet model.
3. Analyze your results.
4. Gain experience with PyTorch, a popular deep learning framework.

### Dataset

The Animals-10 dataset contains about 28K medium quality animal images belonging to 10 categories: dog, cat, horse, spider, butterfly, chicken, sheep, cow, squirrel, elephant. The main directory is divided into folders, one for each category. Image count for each category varies from 2K to 5 K units. Each of the image belongs to one of the 10 categories. In this assignment, you will train classifiers to classify images to one of the 10 categories. You need to divide your dataset into train, validation, and test sets. If the computation time is a problem, you can use a subset of this dataset. However, make sure that your training set is at least 500 images ( 50 images per class). The validation and test sets should include at least 10 images per class, a total of 100 images each. Note that the more data you use, the better the learning will be. **Explain how you arrange your dataset into train, validation, and test and write how many images are there in the sets. [5 pts]**

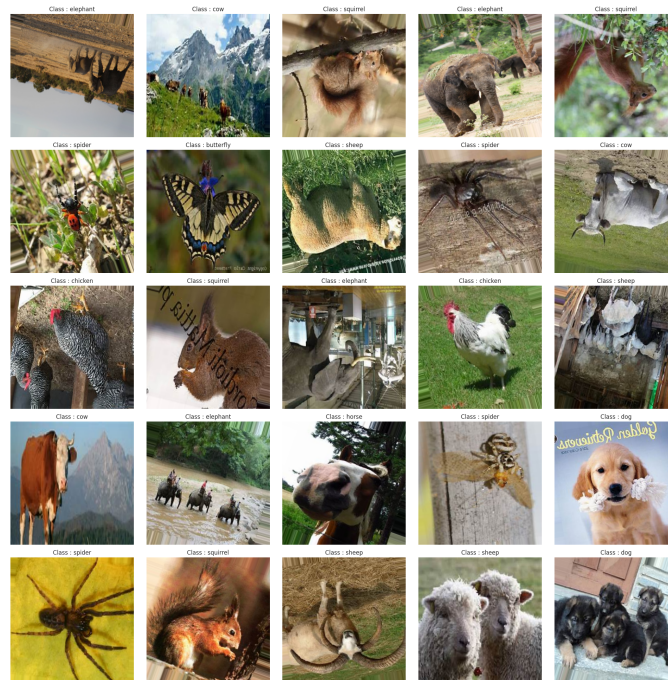


Figure 1: Animal-10 Class Samples. [1]

## PART 1 - Modeling and Training a CNN classifier from Scratch [55 pts]

In this part, you are expected to model a CNN classifier and train it on the dataset provided. You should first define the components of model design. Your model should have 6 convolution layers and two fully connected layers. After the first layer, add Max Pooling layer.

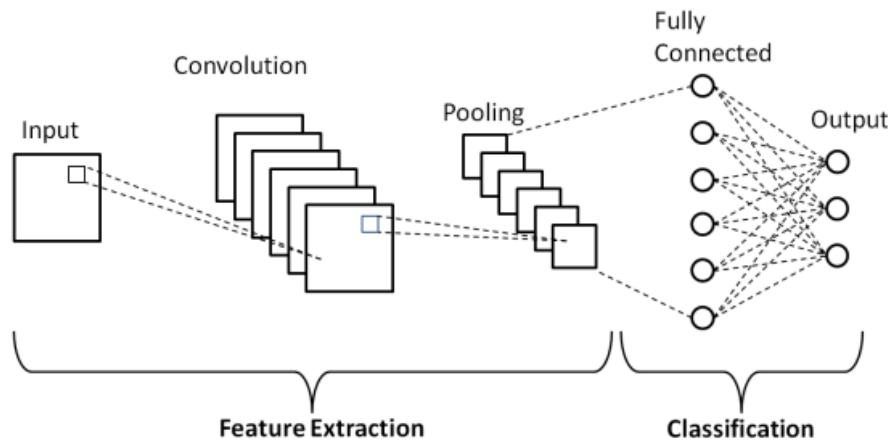


Figure 2: Basic CNN Architecture [2].

- Give parametric details with relevant Pytorch code snippets; number of in channels, out channels, stride, etc. Specify your architecture in detail. Write your choice of activation functions, loss functions and optimization algorithms with relevant code snippets. [2.5 pts x 2 = 5 pts]
- Explain how you have implemented Convolution and Fully-Connected Layers and give relevant code snippets. [5 pts]
- You can apply augmentation on images to obtain better accuracy.

### Training and Evaluating your model

You will set epoch size to 100 and evaluate your model with the chosen learning rate and batch size. Explain how you calculate accuracy with relevant code snippet. Moreover for each of the points below add relevant code snippet to your document.

1. Draw a graph of loss and accuracy change. [5 pts x 2 = 10 pts]
2. Select your best model with respect to validation accuracy and give test accuracy result. [2.5 pts x 2 = 5 pts]
3. Integrate dropout to your best model (best model should be selected with respect to best validation accuracy. In which part of the network you add dropout and why? Explore four different dropout values and give new validation and test the accuracy. [5 pts x 2 = 10 pts]
4. Plot a confusion matrix for your best model's prediction. [5 pts x 2 = 10 pts]
5. Explain and analyze your findings and results. [5 pts x 2 = 10 pts]

## PART 2 - Transfer Learning with EfficientNet [40 pts]

EfficientNet, is a highly effective convolutional neural network (CNN) architecture. **Here** is an explanatory video to understand the logic behind the architecture. EfficientNet-B0 model is one of the standard EfficientNet variants consisting of around 66 layers, including the stem convolutional layer, repeated blocks, and head layers. In the assignment part 2, you will be training the given dataset using the pre-trained EfficientNet-B0 model, which is available at Pytorch. However, it's important to clarify that the fine-tuning process involves adapting the pre-trained model's weights to the dataset given while keeping the initial weights from its starting point. Therefore, while the pre-trained model is based on ImageNet dataset, it will be fine-tuned on the given dataset to improve its performance for the specific classification task at hand. To train the pre-trained model, you need to freeze all the layers before training the network, except the FC layer. Consequently, the gradients will not be calculated

for the layers except the ones that you are going to update (FC layer) over the pre-trained weights. Nevertheless, since the number of classes is different for our dataset, you should modify the last layer of the network, which the probabilities will be calculated on. Therefore, the weights will be randomly initialized for this layer to adapt to the new classification task.

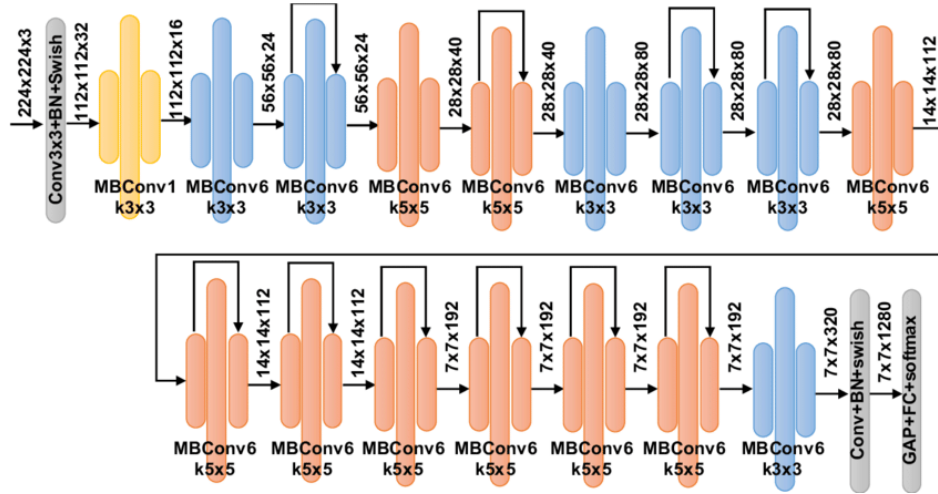


Figure 3: EfficientNet-B0 Model [3].

1. What is fine-tuning? Why should we do this? Why do we freeze the rest and train only FC layers? Give your explanation in detail with relevant code snippet. [5 pts]
2. Explore training with two different cases; train only FC layer and freeze rest, train last two convolutional layers and FC layer and freeze rest. Tune your parameters accordingly and give accuracy on validation set and test set. Compare and analyze your results. Give relevant code snippet. [20 pts]
3. Plot confusion matrix for your best model and analyze results. [5 pts]
4. Compare and analyze your results in Part-1 and Part-2. Please state your observations clearly and precisely. [10 pts]

## What to Hand In

Your submission format will be:

- README.txt (give a text file containing the details about your implementation, how to run your code, the organization of your code, functions etc.)
- code/ (directory containing all your code)
- report.pdf

Archive this folder as **b<studentNumber>.zip** and submit to <https://submit.cs.hacettepe.edu.tr>.

## Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss with your classmates about the given assignments, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered as a violation of academic integrity. Please note that the former condition also holds for the material found on the web as everything on the web has been written by someone else.

## References

- 1 <https://www.kaggle.com/datasets/alessiocorrado99/animals10/data/>
- 2 <https://medium.com/techiepedia/binary-image-classifier-cnn-using-tensorflow-a3f5d6746697/>
- 3 [https://www.researchgate.net/figure/The-EfficientNet-B0-general-architecture\\_fig2\\_348470984/](https://www.researchgate.net/figure/The-EfficientNet-B0-general-architecture_fig2_348470984/)